Latent Causal TAMP

Implementation and experimental guide v1.6 · 2D / 3D

Minimal workspace, executable fixtures, scene-backed visual gates

15 October 2026

Contents

1. Files and ownership: deliberately small **2**

2. Configuration registry, not configuration machinery **3**
2.1 2D robot candidate profile — explicit but unverified . . . 4
2.2 3D cupboard candidate profile — block unpinned assets . . . 5

3. Unbreakable experiment and visual contract **7**

4. Mandatory development gates (no dense shortcut) **8**
4.1 Repaired fixture identities . . . 8

5. Stage registry: one implementation stage at a time **9**
G0 — scene, records, visual foundations . . . 9
G1 — 3R FK/IK, continuous robot phases and first valid nominal skill . . . 9
G2 — frozen obstruction and local causal effects . . . 10
G3 — analytic endpoint assignment, A0 and full move-once order verifier . . . 10
G4 — Boltzmann proposal engine and compact dataset generator . . . 10
G5 — frozen visual features, factor learning and joint fine-tuning . . . 11
G6 — full ablation gate before causal/perception claims . . . 11
G7 — 3D cupboard geometry, assets and mandatory low-density dev gates . . . 11
G8 — 3D full-order verification, reobservation and density challenge . . . 11
G9 — 3D data, separate training and matched evaluation . . . 12
G10 — multi-action scripted long-horizon integration . . . 12

6. Small fixtures: exact worked tests **12**
6.1 `J02` terminal-label sanity pool . . . 12
6.2 `T07/F8` finite domain exhaustion . . . 12
6.3 `T08` real sixth executable relocation . . . 13
6.4 Counterfactual causal visualization . . . 13

7. Evidence, instrumentation and exact acceptance record **13**

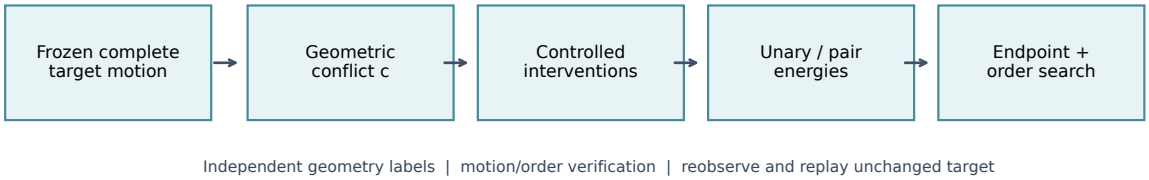
8. Experimental table and claims release **14**

9. Final consistency matrix and release checklist **14**

10. Scope guardrails **15**

Implementation authority. This guide is the stage-by-stage companion to *Latent Causal Infeasibility — Methodology v1.4*. It replaces guide v1.5 for starting a **new** repository. The methodology defines scientific semantics; this guide pins candidate implementation defaults, files, tests, figures, stages and acceptance gates. Reported robot outcomes remain `NOT_RUN` until independently measured. Do not use older master documents' 29-case plan, original residual-latent claims, Slot Attention, action-phase classification heads or 2D disc-only successes as substitute architecture.

Principle: make the complete articulated robot work in **small**, verifiable development cases before launching dense benchmarks. The learned model is not permitted to rescue an invalid reference trajectory or an incomplete collision oracle. A failure that cannot be separated into endpoint, order, motion, controller and numerical categories is not ready for training.



The stage-level architecture. Geometry and intervention evidence are privileged; endpoint predictions are learned; execution remains independently verified.

1. Files and ownership: deliberately small

A flat root-level module layout; create each file **only when its first stage arrives**. No source-package hierarchy, generic utilities module, trainer framework or plugin registry. Keep ordinary modules **roughly 120–280 logical lines**; if a file exceeds ~300, split one clearly separated responsibility with a recorded reason rather than adding abstractions.

```
latent-causal-tamp/
  README.md                 # one-page start, run/gate instructions
  requirements.lock         # actually tested versions; no guessed CUDA pin
  config.toml               # only 2D suite values
  cupboard.toml             # only 3D suite values (created G6)
  fixtures.json             # reproducible cases, roots and lineages
  scene.py                  # stable IDs, cells, state, snapshot, config validation
  viz.py                    # all scene overlays, filmstrips, GIFs
  geometry.py               # exact local/terminal checks + 3R FK/IK/collisions
  motion.py                 # fixed paths, phase logs, arm planner adapter
  interventions.py          # matched state edits, changed-field audit
  solve.py                  # A0 matching/k-best, Gibbs, assignment scoring
  verify.py                 # move-once order and full-robot motion verification
  data.py                   # labels, tensors, loader allowlist and splits
  model.py                  # frozen features / shared local factor nets
  learn.py                  # factor & joint training, checkpoints
```

```
evaluate.py                # paired comparison and confidence intervals
run.py                     # the sole command dispatcher
assets/                    # pinned cupboard MJCF + xArm7/gripper asset subtree
tests/                     # small tests for each stage
runs/                      # flat stage outputs
```

Do not generate empty future source files. `scene.py` never imports training or simulator visuals; `geometry.py` accepts immutable scene queries; `motion.py` owns actual paths; `solve.py` consumes factor tables and candidate maps; `verify.py` owns physical witness search; `data.py` separates observations and oracle labels; `run.py` wires actions but is never imported elsewhere. The 3D backend is a compact backend function set, not a second parallel stack.

Four data records: `Scene` (root/variant, stable IDs, poses, robot/attachment state, supports, legal spots, selected immutable path hash); `FactorTables` (unary, unordered pair, per-object index-to-spot maps, masks); `QueryResult` (three-valued status, evidence scope, reasons, budgets, calls and hash); `RepairState` (achieved scene, robot pose, cumulative moved-object IDs, episode ID, active action hash). Use plain dataclasses + JSON/NPZ, no database. Store `oracle_only` separately; the learner loader uses an allowlist.

One CLI: `python run.py {scene,check,solve,generate,features,train,evaluate,replay,task} --config ... --case ... --out runs/...` (a command becomes active only in its stage). One **flat** output directory per run containing `config.json`, `result.json`, `scene.json`, `overview.png`, optional `trace.gif`, `arrays.npz`, `metrics.csv`, `manifest.json` and, if training, `checkpoint.pt`. No per-frame nested directories. All figures derive from actual logged states, never artistic placeholders presented as results.

2. Configuration registry, not configuration machinery

Use Python >=3.11 `tomllib` and one per-suite TOML file. CLI may override **one named ablation variable**, and the resolved settings must be serialized, hashed and validated. Reject unknown config keys. Generate separate seeds for roots, nominal planner, interventions, rendering, model initialization and inference. Float64 for geometry and tolerances; float32 for networks. Pin simulator/library versions **after tests pass**, recording GPU/device/CUDA versions separately; do not reuse an incompatible historical Torch build.

Global name	Value / convention	Must prove or record
`pitch_m`, `rows`, `cols`	0.08, 8, 8	64 unique supported base cells; row increasing with world +y
`radius_m`, `margin_m`	0.024, 0.005	Distinct cell pitch 0.08 > required 0.053 m
`num_obstacles_2d`	35 challenge; 0–2 direct blockers **also** mandatory	Root total 30–40 in principal dense suite
`num_targets`	1–3, one active per query	Others stationary collision context
`margin_policy`	`UNINFLATED_ORACLE_DELTA_ONCE`	No double inflation in learner and checker

Global name	Value / convention	Must prove or record
`label_policy`	1 compatible; 0 incompatible; unknown masked	Definite zero trumps other unknown factors
`episode_budget`	increment from number already moved; cap <= N	Count distinct moved IDs, not joint states or attempts
`domain_policy`	`ALL_SUPPORTED` default	`RESTRICTED` only with explicit spot list and paired all-supported control
`shape`	identical discs (2D top view), upright cylinders 3D	No heterogeneous shapes claimed
`status`	verified / scoped certified impossible / unresolved	`OUT_OF_SCOPE` separate flag

2.1 2D robot candidate profile — explicit but unverified

Use these as **initial values in `config.toml`**, then freeze a *tested* `robot_profile_hash` after G1; a proposed numeric setting is not a measured configuration:

```toml
[suite]
name = "planar_3r"
rows = 8
cols = 8
pitch_m = 0.08
obstacle_radius_m = 0.024
margin_m = 0.005
num_obstacles = 35
num_targets = 1
coordinate_frame = "world_xy_y_back"
domain_policy = "ALL_SUPPORTED"
margin_policy = "UNINFLATED_ORACLE_DELTA_ONCE"

[arm2d]
base_xy_m = [-0.14, 0.32]
link_lengths_m = [0.42, 0.34, 0.20]
link_full_widths_m = [0.018, 0.016, 0.014]
joint_limits_rad = [
  [-3.14159265, 3.14159265],
  [-3.14159265, 3.14159265],
  [-3.14159265, 3.14159265]
]
q_home_rad = [0.0, 0.0, 0.0]       # provisional: MUST PASS collision / reach checks
base_radius_m = 0.018              # provisional colliding fixed base
joint_housing_radius_m = 0.012     # provisional capsule/joint geometry
self_collision_exclusions = [[0, 1], [1, 2]]

[gripper2d]
```

```
model = "two_finger_side"
palm_length_m = 0.028              # provisional
palm_full_width_m = 0.034          # provisional
finger_length_m = 0.050            # provisional
finger_thickness_m = 0.007         # provisional
open_gap_m = 0.070                 # must exceed 2r plus grasp allowance
closed_gap_m = 0.046               # contact model; not a grasp-force claim
tcp_at_palm_tip = true
attachment_pose_policy = "validated_contact_at_closure"
contact_pos_tolerance_m = 0.001
release_xy_tolerance_m = 0.002

[geometry]
numeric_margin_band_m = 0.0002
interval_max_step_rad = 0.02    # sampling aid ONLY; not certification
interval_certifier = "analytic_or_conservative_subdivision"

[planner2d]
algorithm = "RRT_CONNECT_WITH_FULL_BODY_CALLBACK"
seed = 41
max_nodes = 20000
max_wall_seconds = 30.0
reference_bank_target_K = 3

[inference]
budget_cap = 6
proposal_chains = 8
proposal_sweeps = 100
proposal_temperature = 1.0
max_candidates_to_verify = 200
```

Crucial G1 profile rejection rule: the straight configuration `q_home=[0,0,0]` may extend into or outside the table and collide depending on current robot geometry. If it fails, choose and freeze a valid collision-free **numerical** home via the explicit FK/IK/self-collision test; update this TOML and fixture hashes *before generating any labels*. Do not claim the provisional home is executable. Record gripper aperture interpretation (tip-to-tip **clear gap** vs actuator stroke), orientation sign, jaws' mounting frames, grasp contact exceptions, start/release/retreat/connector pose tolerances, contact policy and robust real interval verifier. No physically meaningful default for controller friction can be assumed from this synthetic 2D representation.

2.2 3D cupboard candidate profile — block unpinned assets

```
[suite]
name = "cupboard_xarm7"
rows = 8
cols = 8
pitch_m = 0.08
```

```
obstacle_radius_m = 0.024
obstacle_height_m = 0.075
num_obstacles = 8
num_targets = 1
margin_m = 0.005
domain_policy = "ALL_SUPPORTED"

[cupboard]
interior_width_m = 0.64
interior_depth_m = 0.64
support_top_z_m = 0.005
roof_underside_z_m = 0.30
side_wall_thickness_m = 0.010    # proposed; verify inner dimensions
back_wall_thickness_m = 0.010
roof_thickness_m = 0.010
front_open_y_m = 0.0

[carry3d]
mode = "ATTACHED_SUPPORT_CONTACT_PLANAR"
object_center_z_m = 0.0425
max_z_deviation_m = 0.001
max_upright_tilt_rad = 0.035
allow_yaw = true
allow_vertical_overflight = false
support_contact_during_carry = true

[robot3d]
asset_revision = "UNPINNED_BLOCKING_G6"   # replace with verified commit/hash
mjcf_path = "assets/robots/xarm7/model.xml"
gripper_asset_revision = "UNPINNED_BLOCKING_G6"
base_mount_xyz_m = [0.0, -0.25, 0.0]      # proposal only: fit physical robot
base_mount_quat_wxyz = [1.0, 0.0, 0.0, 0.0]
q_home_source = "validated_asset_home"
tcp_site_name = "UNSET_BLOCKING_G6"
side_grasp_family = "UNSET_BLOCKING_G6"
tool_to_object_transform = "UNSET_BLOCKING_G6"
controller_or_replay = "KINEMATIC_MJ_FORWARD"
attachment_assistance = false

[verification3d]
continuous_joint_interval_policy = "CONSERVATIVE_SUBDIVISION"
contact_pair_table_version = "G6_TO_PIN"
margin_numeric_band_m = 0.0002
```

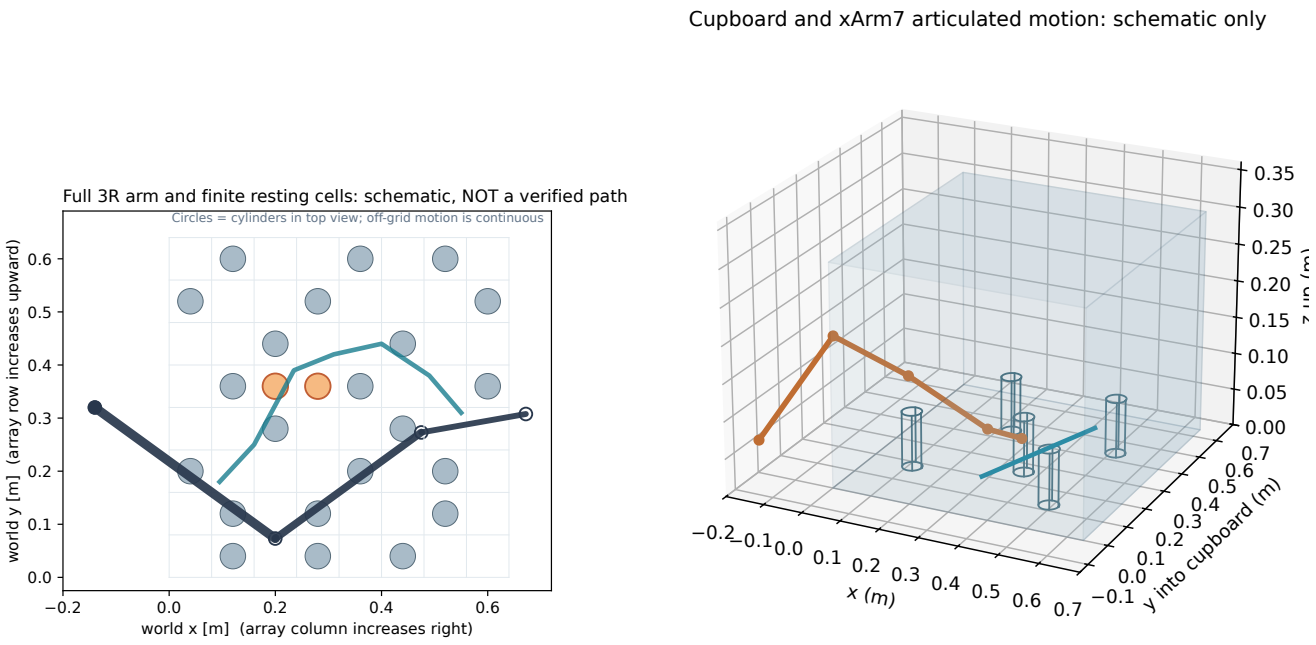
`UNSET`/`UNPINNED` values are intentional **blocking validation keys**, not silent defaults. These cannot be correctly invented from a PDF. At G6 pin actual MJCF + meshes, gripper body/TCP frame, robot base fixture pose, joint names/limits, side grasp / tool-to-object relative pose, home q, collision excludes,

allowed contacts, controller/replay mode, conservative collision subdivision and geometry tolerance. Confirm via MuJoCo on the actual assets, then hash **all** assets and output the fully resolved config. The corpus must not be labeled `FULL_ROBOT_VERIFIED` until that gate passes.

3. Unbreakable experiment and visual contract

For every stage the PNG/GIF provides **three synchronized views** where relevant: (1) actual observed scene and full arm robot pose, with clean learner RGB saved separately; (2) current candidates, local/paired predictions **and** independent oracle labels, or causal per-ID collision contributions; (3) actual tested motion path or precise rejection/unknown interval. A 3D stage also shows front/side section, uprightness and object $z(t)$ trace, cupboard roof/walls and support-contact state. In each figure/GIF show units, x/y/z axes versus array row/column, scene/root/variant IDs, original γ_0 hash, motion scope, current gripper phase, moved history, candidate physical spot IDs, failed collision pair and planner search cap. **Every object stays visible** in failure and replay views.

Use an unchanging axis limit and camera per comparative panel. Do not clip either finger/tool. Color plus written label distinguish oracle valid, invalid and unresolved. A training-loss-only figure does not pass the stage. When no verified witness exists, a schematic must say `SCHEMATIC/NOT_RUN`, not `SUCCESS`. Keep legacy translating-disc pictures solely in a clearly marked diagnostic appendix, not among primary 3R results.



Side-by-side articulated geometry schematics (NOT_RUN): 2D robot and supported resting grid (left); 3D cupboard, walls, roof, cylinders and xArm7-inspired kinematic chain (right). Robot paths and contact dynamics are not yet verified.

Minimum per-stage artifacts: `overview.png`, `result.json` with status/evidence reasons and exact hashes, `arrays.npz` when numeric state evolves, and `trace.gif` for motion/intervention/sampling/repair. For the joint model, include energies and oracle factor masks; for causality, per-ID collision contribution graph plus total action-success sequence (not just the unchanged zero/one scalar). For failed moves, show the **first** violating pair/segment and whether it is a definite collision or numerical/planner

unresolved. Run stage video/GIF must replay *saved* trajectories rather than visually interpolating straight lines between legal endpoints.

4. Mandatory development gates (no dense shortcut)

Gate	2D complete articulated robot	3D full xArm7 + cupboard	Why it exists
Dev-0	Target side-grasp/carry/release in uncluttered support	Same with permitted support contacts	FK, attachment and stage machine
Dev-1	One frozen-sweep blocker, one independently verified relocation	Same	Causal response and basic recourse
Dev-2	Two blockers with two competing destinations; independent oracle labels	Same	Pair coupling and actual slot identity
Dev-3	Dependency on order/access with nontrivial actual robot path	Same (cupboard access)	Separates endpoints from execution
Dense	30–40 obstacle root, five independently necessary frozen action blockers	Pilot 6–10 only when constructible; densify later if supported	Research challenge, not bring-up unit

Every fixture retains the full arm, fingers, selected body, contact and connectors. The difficulty gates are separate roots/splits, not "five-blocker cases simplified by removing four objects without relabeling." If a dense no-lift/3D five-blocker case is impossible to instantiate with the selected geometry, record `CONSTRUCTION_UNAVAILABLE`, adjust *published* fixture dimensions and revalidate, or omit it from reported success; never claim phantom robot outcomes.

4.1 Repaired fixture identities

Retain `T00-T11` for **dense** families and `C00-C07` for 3D pilot as previously named, but do not reuse dense `T02` for the two-blocker training pool. Introduce `J02` for the independently verified two-direct-blocker teaching/control scene. Let `D0`, `D1`, `D2`, `D3` be simple development fixtures and `C-D0` etc. their 3D equivalents.

- `T00`: dense reference physically populated **before** full 3R trajectory planning. Store action bank and five-edit blocker lineage.
- `T03`: actual arm-sweep five-direct-blocker attribution. Each of the five remains a direct blocker unless moved; all one-edit twins share hash and root.
- `T04`: candidate endpoint overlap. The two candidate destinations are individually safe but simultaneous same cell is invalid.

- `T05`: occupied *preferred* target spot; label the assignment invalid, not "requires a sixth terminal move." Show an unrestricted alternative if one exists.
- `T07` / `F8`: restricted legal-cell domain **only**; prove infeasible by matching/Hall or complete endpoint oracle. Publish all-supported paired twin with valid restoration.
- `T08`: five required direct blockers with **at least six verified executable moves**, only if order/access makes a sixth necessary under declared scope; a failed five-move planner budget is unresolved, not proof.
- `T09`: terminal-valid but order-unverified/impossible under restricted move-once semantics; keep statuses distinct.
- `T10`: six genuinely independent direct blockers; if six-move witness exists, lower+upper close optimum.
- `C04`: 3D side-grasp reach/access case uses the **full xArm7**, not a planar 3R surrogate.
- `J02`: 2 blocker teaching pool, with both $A \to P, B \to Q$ and swapped assignment labeled positive **only after** independent action-unary/workspace/pair checks.

5. Stage registry: one implementation stage at a time

The **only** milestone IDs are `G0`–`G10`. Do not mix older methodology `M0..M7` with guide Phase 0..8, and never refer to "M6" without historical qualifier. The `G` gates are sequential unless a noted branch is independent. For each, implement only listed modules, run tests, produce visual, issue the mandatory stage summary and **STOP** before starting the next gate.

G0 — scene, records, visual foundations

Files: `scene.py`, `viz.py`, `run.py`, `config.toml`, `fixtures.json`, `tests/test_scene.py`, `README.md`.

Build stable object IDs and 64 physical cell IDs; initial 35-obstacle illustrative scene; current per-object `stay=0` map; fixed robot-base marker; actual target and masks in world coordinates; JSON/NPZ snapshot round trip and deterministic hash. The figure must draw full illustrative 3R arm and two fingers within axes (but mark **NOT_RUN**). Assert row-to-world and raster-y transforms, area/equal aspect, no duplicate physical cells, source/candidate no-clone interpretation. **Pass:** two builds from one seed produce equal scene hash; permutation of object list changes neither physics nor plot; unknown TOML keys fail immediately.

G1 — 3R FK/IK, continuous robot phases and first valid nominal skill

Add: `geometry.py`, `motion.py`, `tests/test_arm2d.py`.

Install the selected 2D planning dependency **NOW**, not Phase 6: e.g. packaged RRT-Connect/PRM with collision callback plus a small 3R adapter. Use two elbow IK branches, joint bounds, full capsule/finger/palm/base body shapes, self-collision masks, approach/contact/attachment/carry/release/retreat modes and connector tests. Validate FK/IK closure at sampled configurations, reproducible robot home, grasped-object transform under carry, first-collision reporting and swept interval near a thin obstacle. Use `D0` (no clutter) then a sparse occupied scene as references. **Pass:** a complete full-arm side-grasp nominal trajectory exists and replays from exact stored q/g/m; all sampled/certified path segments meet declared margin, no clipped gripper in plots, no decoder/ML needed. Freeze **tested** 3R config hash here.

G2 — frozen obstruction and local causal effects

Add: `interventions.py`, `tests/test_interventions.py`.

Plan/validate γ_0 with its background already present; freeze full arm/tool/object envelope; create matched one-edit descendant moving an existing object into the sweep; independently test action conflict and per-entity contributions under diagnostic removal/relocation. Save intervened snapshot lineage and target hash; all other state fields unchanged. Use `D1` then `D2`, not 35-object five-blocker first. Implement oracle margin exactly once and three-valued factor aggregation. Test object-contact exceptions, support-valid resting placements, symmetric pair nonoverlap, and unrelated edits. **Pass:** positive direct blocker found, verified correction clears *unchanged* γ_0, another blocking object can leave global failure at 0 while its own contribution vanishes. Figure must show fixed trace, per-entity delta and overall action status together.

G3 — analytic endpoint assignment, A0 and full move-once order verifier

Add: `solve.py`, `verify.py`, `tests/test_matching.py`, `tests/test_verify.py`.

Use SciPy's linear assignment/min-cost flow (or equivalent declared deterministic matching solver) for initial identical-grid terminal A0; include all objects even nonblockers, 0 stay cost, 1 relocate cost, one-to-one physical cells and no-good k-best assignment enumeration. A finite fully enumerated small-domain cross-check is mandatory. For general pair-unsafe domains use a discrete constraint solver or search with pair oracle; **do not claim matching exact outside its assumptions**. Take endpoint solutions through every 3R phase and robot-state connector/replay using current state. Search alternate orders **and alternate endpoint assignments**, not just the first matching. Compare 5 direct-blocker lower bound and possible execution upper bound separately.

Regression: in a reference-to-failure identical-disc all-supported case, A0 returns exactly $|D|$ terminal moves and moving those objects to original reference cells passes terminal geometry. Build `T07` restricted-domain control with finite Hall or complete-search no-solution certificate plus an all-supported twin. Test movement of a destination occupant can change *order/motion* need even though minimum terminal cardinality is unchanged. **Pass:** no misleading six-terminal-move or unrestricted-F8 labels; all claimed executable witnesses replay the **same** original action with persisted ID/hash. An unfinished search gives `UNRESOLVED`.

G4 — Boltzmann proposal engine and compact dataset generator

Add: `data.py`, `tests/test_data.py`.

Implement soft analytic factors, categorical Gibbs with $b - |H|$ budget, random sweep order, multiple seeds, temperature variation, dedup, proposal count, acceptance rate and independent oracle ranking. Use same solver domain and physical verifier as A0. Add lineage-aware generation: 0/1/2 blockers first, later 20 audited dense *3R verified* roots, 300 independent root pilot and conditional 1,000 **root** expansion; save root count, sibling-query count, pool size and solver cost separately. Group every variant, trajectory sibling and render by root split. **Pass:** small enumeration matches terminal oracle exactly; A0 and Gibbs receive exactly equal information and call-budget accounting; loader banishes privileged oracle fields.

G5 — frozen visual features, factor learning and joint fine-tuning

Add: `model.py`, `learn.py`, `tests/test_model.py`.

Freeze DINOv2 ViT-S/14 (or explicitly declared available equivalent), pool image patch features under GT masks, project to width 128; candidate-relative geometry and envelope sampled via pointwise MLP/pooling. Use shared two-layer width-128 MLPs, symmetric pair logits and softplus(-logit) energies. Preserve the discrete physical candidate map. Numerical starts: AdamW LR 10^{-3}, weight decay 10^{-4}, $T_{\text{train}} = 1$, $\lambda_J = 1$, mixed valid/invalid joint pool $M \approx 32$, 3 independent model seeds. These are **starting settings**, not performance claims. Optimize local BCE first, then local BCE plus joint finite-pool loss. Use `J02` with verified two-blocker positives and negatives (or dense five-blocker positive pools with three remaining blockers' destinations explicitly fixed). **Pass:** local factor oracle audit, pair permutation exactness, missing-mask rejection, no leakage, joint loss sign/gradient finite-difference test, model checkpoint data/split/config hash validation. Show actual scene, ghost factor probabilities, terminal oracle labels and validation repair witness—not only loss curves.

G6 — full ablation gate before causal/perception claims

Add: `evaluate.py`, `tests/test_evaluate.py`.

Run A0–A4 minimum on the *same* held-out roots and solver budgets. Run A4-L, A5-I, **A5-R retrained**, A6, A7, A8, A9, **A10 matched-observation vs matched-intervention**, and A11 (all-supported vs restricted) as research claims require. A5-I (remove pairs only at inference) and A5-R (retrain without pairs) answer **different** questions. For the final visual model compare both local-only and joint checkpoints. A8 must not simultaneously remove coupled examples **and** joint loss if you intend to isolate intervention data. A10 balances roots, labels, budgets and family coverage. Bootstrap over independent roots; capture false local factor, sampler miss, endpoint invalid, motion unresolved, connector fail and reobserved deviation. **Pass:** no success percent hides an unresolved denominator; an analytic A0 win is reported accurately. Keep test held out throughout tuning.

G7 — 3D cupboard geometry, assets and mandatory low-density dev gates

Add: `cupboard.toml`, pinned `assets/robots/xarm7/` and cupboard geometry under `assets/` (no new 3D training framework); `tests/test_cupboard.py`.

Pin exact xArm7/gripper asset revision/license, TCP frame, mount transform, cupboard walls/roof thickness, support height, closed/open aperture, side-grasp family, tool-to-object transform, contact schedule, controller/replay mode and conservative interval verification. Reject any `UNSET` blocking field before accepting results. Build full 8×8 square grid, 6–10 obstacles only for pilot and 1–3 targets. **First execute `C-D0`, `C-D1`, `C-D2`, `C-D3`** with actual xArm7 and side-gripper, no lift. Enforce object center $z = 0.0425$ m for 0.075 m cylinders with support top 0.005 m: bottom remains contacting support through attached transport; uprightness and yaw convention are checked **between** IK nodes. **Pass:** robot can side-grasp with allowed support contact but never intersects cupboard, other objects or self; full-arm 3D configuration trace + top/side view + $z(t)$ + tilt(t) plots; robot links not incorrectly required to be at support-plane z.

G8 — 3D full-order verification, reobservation and density challenge

Add only: new fixtures/tests/replay paths within existing `verify.py`, `motion.py` and visualization. Instantiate `C00-C07` after development success; preserve true xArm7 checks. If five direct blockers cannot

be constructed, report case unavailable, do not promote a sparse private test into a dense benchmark. Search order and endpoint alternatives with full xArm7, support contact, no vertical overflight, connector to exact original start and same frozen target replay. After first physical/sim move record achieved pose and new observations; history freezes moved IDs to stay at actual pose. **Pass:** successful cases replay twice within pose/height tolerances and unchanged hash; any blocked connector or second move of prior ID rejects move-once success; failed IK under finite budget remains unresolved.

G9 — 3D data, separate training and matched evaluation

Reuse: `data.py`, `model.py`, `learn.py`, `evaluate.py`, `viz.py`, suite config switch. No new code hierarchy. Use 20 audited cupboard roots, 300 independent roots (210/45/45) and optional expansion to 1,000 **roots** only if geometry/verification throughput warrants. Show actual separate root/query/pool counts. Audit narrow-gap sampling and whether cylinder appearance adds value. Train 3D separately first; 2D-to-3D weight transfer is an **additional** study and never assumed. Replay examples and run A0–A4 plus matching targeted ablations. **Pass:** same terminal label and evidence semantics as 2D, no 2D-only surrogate authority, all reported 3D cases have real asset/config hashes.

G10 — multi-action scripted long-horizon integration

Reuse: all existing modules with `run.py task` and a small `tests/test_task_sequence.py`.

Construct a_1, a_2, a_3 with explicit **trajectory freeze times**. Test both (i) pre-frozen all actions and (ii) just-in-time freeze at current scene, if both compared. Induce one blockage and repair; execute; preserve achieved object poses into later action. Include a scene where the first repair clears its local action but obstructs a **pre-frozen** later action. History persists from first intervention to completion/abandonment of that target episode, then resets for the **next** original target episode only. Measure local repair success, complete 3-action success, number of planner/repair invocations and suffix interference; show full timeline with frozen hash per target action, achieved states and causal contributions. **Pass:** next action never planned from a stale pre-repair state under JIT protocol; pre-frozen test never silently replans; honest reporting distinguishes local success from complete task success.

6. Small fixtures: exact worked tests

6.1 `J02` terminal-label sanity pool

Define only **two direct blockers** A, B and two supported safe physical destinations P, Q; other objects may be present as irrelevant, but no remaining direct blockers. Independently compute $y^a_A(P), y^w_A(P), y^a_B(Q), y^w_B(Q), y^p_{AB}(P, Q)$ and the swapped counterparts. If all are 1, both $A \to P, B \to Q$ and $A \to Q, B \to P$ are terminal-positive. One-blocker-stay candidates are negative because the other direct blocker still intersects the action. Same destination for both is pair-invalid. Verify a real full-arm two-relocation motion separately; if motion is unresolved, terminal pool labels remain valid and execution status remains unresolved. **Do not label by fixture name or prescribed response.**

6.2 `T07/F8` finite domain exhaustion

Start with the guaranteed five-object all-supported restoration reference root. Create a **new restricted policy** whose published allowed subset has fewer distinct action-safe physical cells than the number of objects needing assignment; show Hall's violating subset U with $|N(U)| < |U|$ or exact complete

endpoint exhaustion. Save the forbidden/enabled physical cell list and a matching solver certificate. Run the **same root** with unrestricted `ALL_SUPPORTED`: it must pass with cardinality $|D|$. Do not call a structurally reachable but motion-inaccessible cell terminal-invalid.

6.3 `T08` real sixth executable relocation

Start with 5 direct current blockers, independently proving the 5-move lower bound. Demonstrate 6 distinct full-arm object relocations with all phases and a final connector/target replay. To claim 6 is an *optimum* (not just a witness), certify no 5-move **executable** sequence under a complete declared search/physical access impossibility proof. Candidate-domain restriction may create a six-terminal **restricted** instance, but must not be described as unrestricted motion-order causation. When finite stochastic motion search merely fails to find a 5-move witness, label `LB=5, UB=6, MINIMUM_UNRESOLVED`.

6.4 Counterfactual causal visualization

For 3 current independent direct blockers, success of frozen target is 0 with all present, remains 0 after moving any **one**, remains 0 after moving any **two**, and becomes 1 only when all 3 no longer intersect the frozen sweep. Plot c_i and d_I after each edit alongside action success; this prevents the mistaken conclusion that each individually necessary blocker is not causal because the global success delta was 0.

7. Evidence, instrumentation and exact acceptance record

Use the same strict status contract:

```
{
  "stage": "G3",
  "fixture": "D2",
  "status": "UNRESOLVED",
  "verification_scope": "PLANAR_ARM_KINEMATIC_SEARCH",
  "root_family_id": "root_0007",
  "episode_id": "ep_0007_a1",
  "target_hash": "sha256:...",
  "robot_profile_hash": "sha256:...",
  "domain_policy": "ALL_SUPPORTED",
  "clearance_source": "UNINFLATED_GEOMETRY_PLUS_DELTA_ONCE",
  "direct_blocker_count": 2,
  "terminal_lower_bound": 2,
  "executable_lower_bound": 2,
  "executable_upper_bound": null,
  "minimality_status": "NOT_CERTIFIED",
  "moved_history": [],
  "motion_search_calls": 430,
  "motion_search_budget_seconds": 30.0,
  "rejection_reason": "ORDER_PATH_SEARCH_EXHAUSTED",
  "target_replay_status": "NOT_ATTEMPTED",
  "simulation_replay_status": "NOT_RUN",
  "physical_execution_status": "NOT_RUN"
}
```

Scope wording must never use `FULL_ROBOT_VERIFIED` to imply a dynamic/controller or hardware execution. Attach a complete kinematic witness file and separate `SIM_REPLAY_SUCCESS` or hardware evidence fields where actually measured. Store actual achievement rather than just commanded endpoints and log deviation. Do not confuse a failed *grasp executor* with a terminal-invalid geometry query.

Smallest stage report, required after every `G` gate: stage/version, newly created/modified files and approximate lines, full resolved config hash, fixtures attempted/passed/failed/unresolved, first failure cause and whether a numerical/asset blocker remains, testing commands/exit codes, paths to scene-backed PNG/GIF/replay, and explicit **STOP** before the next gate.

8. Experimental table and claims release

The **methodology v1.4 Table A0–A11** is canonical. Save per-root per-method rows, not just aggregate bar charts. Required columns: root/family/variant/trajectory ID; domain policy; direct blockers; terminal matching optimum; frozen target success; scope; attempted assignments/unique assignments; first valid terminal candidate rank; motion and order outcomes; executable lower/upper bounds; moved count; verifier calls; feature/factor/sampler/matching/planner times; all failure reasons; sampled seed; checkpoint/split hashes; later-task success when applicable.

Produce a *single matched difficult scene* for A0, A1, A2, A3, A4, plus comparison among geometry-only local, geometry-only joint, **visual local**, **visual joint**, retrained unary-only and pair-inference-lesion. For A10 include observational and intervention-trained examples with same number of independent roots, labeled factor queries and family proportions. For A11, use the same root under all-supported and explicitly restricted domains. A plot should not replace per-query reproducible data.

Report denominator definitions exactly: blocked-repair success on eligible blocked cases only; overall query success on all eligible cases including zero-move feasible ones; counterfactual validity on actually claimed repairs. Unresolved remain in the evaluated denominator, with count; out-of-scope exclusions separately reported. Cluster bootstrap at **root** level. Never claim a zero-resolved-denominator fraction as 100%.

9. Final consistency matrix and release checklist

Invariant	Methodology v1.4	Guide v1.6 enforcement
Frozen action includes full articulated arm	Sections 2–3	G1/G2 pin and hash the full robot phase trace
Single margin application	Section 3	Explicit `margin_policy`, near-margin test in G2
Direct blocker count is unrestricted terminal optimum	Section 7 theorem	G3 A0 restore-to-reference regression
No generic large-grid brute force	Section 7.2	G3 min-cost matching + k-best and scoped alternate solver
Real paired joint-positive labels	Section 9 (`J02`)	G5 independently oracle-checked 2-blocker pool
Contact through entire no-lift 3D carry	Sections 2.2, 8	G7 full pair table and full interval tests

Invariant	Methodology v1.4	Guide v1.6 enforcement
3D carried object z, θ constrained throughout	Section 2.2	G7/G8 height, tilt, yaw and side-view traces
Same candidate count interpretation	Sections 7, 11	G4/G9 distinguish roots, queries, candidates
No conflicting milestone numbering	Section 15	Only `G0-G10` registry
Separated success denominators	Section 12	G6/G9 explicit population counts
Learn pair and intervention effects with matched controls	Section 12	A5-R, A4-L, A8–A10
Actual long-horizon failure from locally valid repair	Section 13	G10 prespecified per-action freeze protocol

Release condition: all ten prior gate checkpoints (G0–G9) and scripted integration gate G10 have machine-readable outcomes with accurate status—not necessarily 100% solution success. Both files use the same reference-grid theorem, domain semantics, contact policy, milestone names, root counting, metrics and ablation identities. The project can *begin at G0 immediately*; robot profile/asset defaults remain explicitly provisional until their required experimental validation, never silently certified.

10. Scope guardrails

Do not introduce robot vertical overflight, online RL, learned contact dynamics, articulated door skills, fluid simulation, full task-plan suffix energies, slot-discovery systems, a sequence decoder, direct IK-only novelty, neural causal-identifiability guarantees, or handcrafted energy targets. Do not build a generic web UI or broad software framework. Do not remove unresolved/failed roots solely because they are inconvenient. Preserve honest geometry–execution separation, the strongest analytic baseline, and scene-backed evidence at every stage.